

In the remainder of this paper, we evaluate several LLM-based question answering settings on this dataset, using the SPARQL query results as gold-standard answers.

4 Evaluations and Results

With all published machine-actionable, symbolic representations of selected parts of review articles in materials science in place, we next perform quantitative evaluations to answer the three research questions introduced at the beginning of the article: (i) how useful domain experts find these ORKG-based comparison representations (**RQ1**), (ii) how purely neural, PDF-based LLM querying compares to the symbolic SPARQL baseline (**RQ2**), and (iii) how neurosymbolic querying over ORKG comparisons performs relative to both (**RQ3**). Together, these evaluations make the case for the importance of *symbolic knowledge representations of scientific knowledge to facilitate neurosymbolic AI*—the central research premise of this paper.

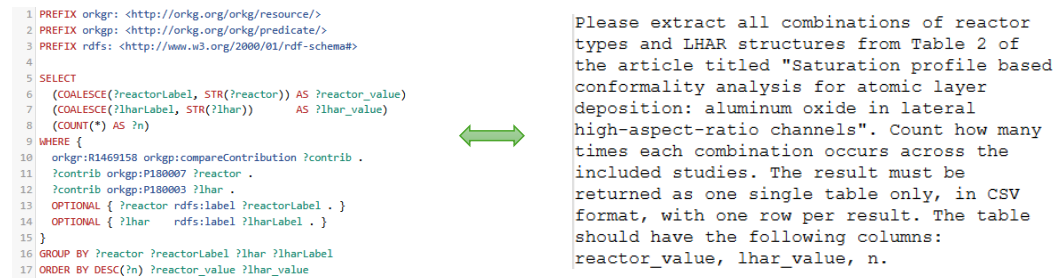


Figure 6: Illustration of a SPARQL query operating over the machine-actionable ORKG comparison graph <https://orkg.org/comparisons/R1469158> and its natural-language equivalent. The latter can be used in LLM-based querying scenarios over different forms of input, such as the article PDF or a serialized version of the comparison. This SPARQL query was introduced in subsection 3.1.

4.1 Evaluation Setup

We begin by describing our evaluation setup: subsection 4.1.1 introduces the experimental dataset, and subsection 4.1.2 outlines the experimental settings.

4.1.1 Experimental dataset

Our evaluations are based on the 33 SPARQL queries introduced in Sec. 3: 19 questions over the ALD comparisons and 14 over the ALE comparisons. Each query runs on one or more ORKG comparisons and returns a machine-generated answer table, which we use as the gold standard.

For the neural (LLM-based) experiments, we then created a natural-language version of each SPARQL query. These natural-language queries explicitly specify (i) which review table in the paper to consult, (ii) which columns in that table to use, and (iii) which columns the answer table should contain and that the output must be formatted as a table. In other words, they convey the same information as the corresponding SPARQL query, but in a form that an LLM can directly follow. An example is shown in Figure 6. We generated these natural-language queries programmatically using *mistral-large*; the Python script is available in our GitHub repository: https://github.com/sciknoworg/ald-ale-orkg-review/blob/main/llm-experiments/nl_query_generator_csv_onetable.py. This yields 33 detailed natural-language queries that map one-to-one to the 33 SPARQL queries and their gold-standard answer tables. Note that these differ from the earlier (section 3), more informal question formulations, which are intended for readers rather than for enforcing a consistent output structure from LLMs. For quick reference, the GitHub repository also lists, for each ALD and ALE review table (including cross-table queries), both the short and detailed natural-language queries and the corresponding SPARQL query links.

For one of the experimental settings (Setting 3, described below), the LLM does not see the full review PDF. (In Setting 2, by contrast, the model is given the review PDF or its extracted text and

must read it to answer the question.) In Setting 3, to bring the neural experiments closer in spirit to the pure SPARQL (symbolic) gold standard of Setting 1, we instead provide the ORKG comparison table itself in a simple, spreadsheet-like format (a CSV file). Each row in this CSV corresponds to one row of the original review table, and each column corresponds to one of the typed properties we modeled in ORKG (for example, material, precursor chemistry, substrate, temperature, etch rate, or selectivity). In other words, the model sees the same structured information that the SPARQL queries in Setting 1 operate on, but now as a flat text table rather than as a figure embedded in a PDF. We obtain these CSV files via the built-in ORKG export function, which also supports additional formats such as $\text{\LaTeX}$, PDF, RDF, and citation metadata. Settings 2 and 3 therefore both evaluate question answering via large language models, in contrast to the direct, structured graph querying in Setting 1. While SPARQL queries are fully deterministic and always return the same result for a given ORKG snapshot, LLMs are probabilistic models: their answers can vary slightly from run to run, even when the underlying evidence (here, the curated comparison tables) remains fixed. High agreement with the SPARQL answers in Settings 2 and 3 should therefore be interpreted as an approximation to the symbolic gold standard, rather than an exact guarantee.

4.1.2 Experimental settings

Setting 1: Symbolic querying of structured comparison tables. In this first setting, we do not use any language models at all. Instead, we directly query the machine-actionable ORKG comparison tables using SPARQL, a structured query language for graphs. For each of the 33 questions, we wrote a corresponding SPARQL query that selects exactly the relevant rows and columns from the curated comparison tables. Because these queries run over structured data and execute deterministically, the resulting answer tables form the gold-standard reference against which we compare all LLM-based settings (Settings 2 and 3).

Setting 2: Neural querying from PDF-based scientific evidence. In this setting, we ask how well large language models can answer the 33 questions when their only evidence is the review article itself in PDF form. Both proprietary and open-weights models receive the detailed natural-language equivalent of the SPARQL query together with the article PDF (or text and tables extracted from it), but they have *no* direct access to the ORKG representations. The models must therefore internally parse the PDF, locate the relevant passages and tables, and produce an answer table in a single neural inference step. By comparing their outputs against the SPARQL gold standard from Setting 1, we can quantify how closely purely neural, PDF-based querying can approach symbolic performance for precise, table-style scientific questions.

2.1 *Proprietary language model (neural) querying.* We first evaluate three state-of-the-art commercially hosted conversational models—ChatGPT-5.1 (OpenAI, release 12.11.2025) [47], Gemini-3-Pro (Google, release 18.11.2025) [50], and Claude-Sonnet-4.5 (Anthropic, release 29.09.2025) [5]. As of the date of this submission (13.12.2025), these systems are among the most capable proprietary large language models. Each model is provided with the detailed natural-language equivalent of the SPARQL query and the PDF of the corresponding review article. This setting tests whether such black-box systems, with their own internal PDF-parsing pipelines, can directly answer precise, table-oriented scientific questions from the PDFs alone. A limitation of this setting is that the parsing and context-integration steps cannot be inspected or modified, and runs cannot be exactly reproduced beyond what the providers expose in their interfaces.

2.2 *Open-weights language model (neural) querying.* To gain full experimental control and reproducibility, we complement the proprietary systems with high-performing *open-weights* large language models. Unlike proprietary models accessed via paid cloud services, these open-weights models expose their parameters and can be downloaded from platforms such as <https://huggingface.co/models> or Ollama <https://ollama.com/> and run locally. Once suitable hardware is available, this removes per-query usage costs and additionally provides full control over model versioning and configuration, strict reproducibility of experiments, and the option to keep all data within a local computing environment (which can be important for sensitive datasets).

For each paper, we first convert the PDF into machine-readable text and tables using `pdfplumber` and our processing script https://github.com/sciknoworg/al-d-ale-orkg-review/blob/main/llm-experiments/pdf_extractor.py. In practical terms, `pdfplumber` reads each PDF page and extracts the visible text plus the grid

structure of tables into plain-text and tabular form. This conversion step is required so that the language models can work with the article content as sequences of text tokens and rows/columns, rather than as a page-layout format optimized for display but whose information is not directly accessible for text processing. We then evaluate five open-weights models: `gemma-3-27b-it`, `llama-3.3-70b-instruct`, `qwen2.5-v1-72b-instruct`, `qwen3-30b-a3b-instruct-2507`, and `mistral-large-instruct`, selected based on their strong performance on recent tabular reasoning benchmarks [11].

For open-weights models, we consider two variants:

- a) *Full-document context (context-injected document QA)*. In this variant, all extracted text and tables from a given paper are concatenated into a single long context and passed directly to the model together with the detailed natural-language query (see `context_injected_doc_qa.py` in our repository). The model must answer using this complete, but potentially noisy, representation of the article. This mirrors how a human might read the entire paper and then answer the question.
- b) *Retrieval-augmented generation (RAG) over PDF segments*. In the RAG variant, we first split the extracted paper into contiguous text segments (roughly a few pages each) and index these segments using a dense semantic embedding model. At query time, we retrieve the most relevant segment(s) for the detailed natural-language query and supply only these retrieved segments, plus the query, to the model (see `rag_pdf_segments_doc_qa.py`). This reduces context length and focuses the model on the parts of the paper most likely to contain the answer, but it can miss information if key table rows or explanations are spread across distant segments.

A detailed, technical version of setting 2 description including our implementation choices for PDF extraction, segmentation, embedding, and retrieval hyperparameters are provided in the Appendix A.

Setting 3: Symbolic-context-augmented neural querying. In this setting, we bring the language models as close as possible to the symbolic baseline by grounding them directly in the machine-actionable ORKG comparison tables instead of the raw PDFs. Concretely, we export each ORKG comparison into a flat, spreadsheet-like format (CSV), as described in Sec. 4.1.1, and provide this CSV together with the detailed natural-language query as input to the models. This data is publicly released [15]. We evaluate the same proprietary and open-weights models as in Setting 2, but here they never see the PDFs or their extracted text. All reasoning is performed over the CSV-encoded comparison tables, which are *identical in content* to those queried via SPARQL in Setting 1; only the query engine changes, from a deterministic symbolic executor to a probabilistic neural model.

We refer to this as *symbolic-context-augmented* querying because the language model is not only given a natural-language question, but also a *symbolic* representation of the evidence: the ORKG comparison table. In AI and data science, “symbolic” typically denotes knowledge represented as explicit symbols and relations (for example, rows and columns in a table or nodes and edges in a graph) that can be queried with formal languages such as SPARQL, rather than as free-running text. In our case, each material, precursor, temperature, etch rate, or EQE value appears in a dedicated column with a well-defined meaning, instead of being embedded in sentences in the PDF. Setting 3 therefore lets us study how well language models can approximate the SPARQL gold standard when their input context includes this structured, symbol-like table representation, rather than only heterogeneous PDF layouts. We do not, however, build a specialised neurosymbolic architecture (such as a separate symbolic reasoning module inside the model); the symbolic information is provided purely as richer input context.

It is important to note that the query engine in Setting 1 is fully deterministic: the same SPARQL query over a fixed ORKG snapshot always returns the same result. By contrast, the neural query engines in Settings 2 and 3 are non-deterministic because their text generation involves random choices; repeated runs can yield slightly different answers, even when the input is unchanged. In Setting 3, the model must still perform the essential operations that SPARQL carries out explicitly: identify the relevant rows in the comparison table (for example, all entries with a given material or precursor), select the correct columns for the requested output, and sometimes compare or combine values across rows. Our evaluation therefore asks to what extent a symbolically grounded neural Q&A system can match the numerical accuracy of the SPARQL baseline (Setting 1), while offering a flexible natural-language front end.